ETL Pipeline Example: Load

Understand how to write and add a load function in an ETL pipeline, using Python to insert transformed data into a data warehouse. Learn to integrate this load step into an Apache Airflow DAG for automated scheduling and execution, ensuring incremental data loading for effective data analysis.

Writing the `load` function in `helper.py`

Now that we have a clean and transformed CSV file, we can focus on the load stage of the ETL pipeline. As usual, we'll write a function called `load` in the `helper.py` file. Later, we'll use that function as a part of the DAG.

The `load` function creates a new dataframe using the clean data and inserts it into the fact table in the data warehouse using a for loop.

helper.py Python

```python
import psycopg2
import pandas as pd

def load():

    # Connect to the database
    conn = psycopg2.connect(host='localhost', port = 5432, database='pos

    # Create a cursor
    cur = conn.cursor()

```

Ask

Run

Writing the load function

We can check the latest data that we just loaded to the fact table by querying the fact table and ordering the results by `comment_date` in descending order.

helper.py Python

```
import psycopg2
import pandas as pd

# Connect to the database
conn = psycopg2.connect(host='localhost', port = 5432, database='postgres

# Create a cursor
cur = conn.cursor()

query = """
select *
from fact_table
order by comment_date desc
```

Run

Checking the newly loaded data in the data warehouse

Adding the `load` function to the DAG

Now that we're done writing the script to load the data and have checked that it works, we can add this script as a part of the DAG. This script will run after the `transform` task in the DAG has been completed successfully. As usual, we'll use a `PythonOperator` to execute this function in the DAG.

grab_latest_date.py Pytho

Ask

```
from airflow import DAG
from airflow.operators.python import PythonOperator
```

```
from helper import extract, transform, load
import datetime

# Create a DAG object
# Set DAG id, start date, schedule interval
with DAG(
    dag_id = 'social_media_pipeline',
    start_date=datetime.datetime(2023,8,22),
    schedule=datetime.timedelta(hours=4)
```

Run

Adding the load function to the DAG

With that, we have completed adding the necessary tasks to our pipeline. The finished workflow looks like this:

DAG workflow

Lastly, we want to check that the new task works as a part of the DAG. Since this task is the last in the DAG, we can simply test the entire DAG all at once. Run the following command to test each task in the DAG, starting from `extract` and all the way to `load`:

Ask

```
airflow dags test social_media_pipeline 2023-08-29
```

If everything is working as expected, we should be able to trigger the DAG using the `airflow dags trigger` command. Once we do that, airflow will run the DAG continuously according to our chosen schedule (once every four hours).

Run the following command to trigger the DAG:

```
airflow dags trigger social_media_pipeline
```

Terminal 1

Terminal

Loading...

And that's it. We now have a schedule-based ETL pipeline that will incrementally load batches of data about the company's users, posts, and comments to the data warehouse for further analysis. Data analysts, data scientists, BI developers, and more can now use the data without affecting the performance of the production database.

Ask

Ask